

AZA 2025/26 - Practical Assignment

Algorithm Complexity Analysis

Submission Date: December 2025

Course: Algorithms and Algorithm Analysis (AZA)

Academic Year: 2025/26

AI Assistance Declaration

In accordance with the assignment requirements (Note 1), I declare that AI assistance (**Claude Code**) was used for the following purposes:

- Code verification and testing
- Complexity analysis review
- Documentation generation
- Bug detection and edge case testing

All core algorithm implementations were written based on the textbook pseudocode and assignment specifications. The AI was used as a verification tool rather than for primary code generation.

Task 1 & 2: Scheduling with Deadlines Algorithm

Problem Statement

Given n jobs with deadlines and profits, find a schedule that maximizes total profit, where each job must be completed by its deadline to earn its profit. Jobs are sorted in nonincreasing order by profit.

Approach 1: Basic Greedy Algorithm (Algorithm 4.4)

Implementation: `scheduling.cpp`

Algorithm Description

1. Sort all jobs by profit in descending order
2. Initialize schedule J with the highest-profit job
3. For each remaining job i :
 - Create tentative schedule K by adding job i to J
 - Check if K is feasible (all deadlines can be met)
 - If feasible, accept: $J = K$; otherwise reject job i

Time Complexity Analysis

Step	Complexity	Explanation
Initial sorting	$O(n \log n)$	Sort jobs by profit
Main loop	n iterations	Try adding each job
Feasibility check	$O(n \log n)$	Sort + linear check per iteration

$$\begin{aligned} T(n) &= O(n \log n) + \sum_{i=1}^n O(n \log n) \text{ for } i=1 \text{ to } n \\ T(n) &= O(n \log n) + n \times O(n \log n) \\ \mathbf{T(n) = O(n^2 \log n)} \end{aligned}$$

Space Complexity

- Storage for n jobs: $O(n)$
- Temporary schedule arrays: $O(n)$
- **Total: $O(n)$**

Approach 2: Optimized with Disjoint Set Data Structure III

Implementation: `scheduling_disjoint.cpp`

Algorithm Description

1. Sort all jobs by profit in descending order
2. Create $d+1$ disjoint sets for time slots $0, 1, 2, \dots, d$ (where $d = \text{max deadline}$)
3. Each set tracks its smallest available time slot
4. For each job i (in profit order):
 - Find latest available slot $\leq \text{deadline}$ using `small(S)`
 - If slot > 0 : schedule job at that time
 - Merge slot with (slot-1) to mark as used
 - If slot = 0: reject job (no available time)

Time Complexity Analysis

With path compression and union by rank:

- Initial sorting: $O(n \log n)$
- Initialize $d+1$ disjoint sets: $O(d)$
- Process n jobs: $O(n \times \alpha(m))$
 - α is inverse Ackermann function (nearly constant)
 - For practical purposes: $\alpha(m) \leq 4$ for all realistic inputs

$$T(n) = O(n \log n) + O(d) + O(n \times \alpha(m))$$

where $m = \min(n, d)$

$$T(n) = O(n \log n) \text{ [dominant term]}$$

Without path compression (textbook DS III):
 $T(n) = O(n \log m)$ [as required by assignment]

Space Complexity

- Disjoint set structure: $O(d)$ where $d = \text{max deadline}$
- Job storage: $O(n)$
- **Total: $O(n + d)$**

Comparison of Both Approaches

Test Results on Table 1.1 Data (7 jobs):

Both algorithms produce identical optimal results:

- Optimal Schedule: [7, 1, 3, 2]
- Total Profit: **170**
- Time slots: 1 → Job7(55), 2 → Job1(40), 3 → Job3(60), 4 → Job2(15)

Metric	Algorithm 4.4	Disjoint Set
Time Complexity	$O(n^2 \log n)$	$O(n \log n)$
Space Complexity	$O(n)$	$O(n + d)$
Scalability	Poor for large n	Excellent
Implementation	Simple	Moderate

Improvement Analysis

The disjoint set approach provides significant asymptotic improvement:

- ✓ **Reduces time from $O(n^2 \log n)$ to $O(n \log n)$ - eliminates n factor**
- ✓ **For $n=1000$ jobs: $\sim 1000\times$ faster in practice**
- ✓ **Key insight: disjoint sets efficiently track available time slots**

- ✓ After scheduling at slot t , merging with $t-1$ makes future searches skip used slots
- ✓ Avoids repeated feasibility checks of the entire schedule

Task 3: Huffman Compression and Decompression

Problem Statement

Design and implement a lossless compression algorithm using Huffman coding. The algorithm assigns variable-length codes to characters, with more frequent characters receiving shorter codes. Implementation must support the full ASCII table (256 characters).

3.1 Compression Algorithm

Implementation: `huffman_compress.cpp`

Algorithm Steps

1. Read input file and build frequency table for all 256 ASCII characters
2. Create leaf nodes for each character with frequency > 0
3. Build Huffman tree using priority queue (min-heap):
 - Repeatedly extract two nodes with minimum frequency
 - Create parent node with combined frequency
 - Insert parent back into priority queue
 - Continue until one node remains (root)
4. Generate binary codes via tree traversal (left=0, right=1)
5. Write compressed file:
 - Header: 256 integers (4 bytes each) = 1024 bytes
 - Total character count (8 bytes)
 - Encoded data (bit-packed into bytes)
6. Replace each character with its Huffman code, pack into bytes
7. Pad final byte with zeros if needed

Time Complexity Analysis

Variables:

- n = number of characters in input file
- c = number of unique characters ($c \leq 256$ for ASCII)

Step	Complexity	Explanation
Build frequency table	$O(n)$	Single pass through file
Create priority queue	$O(c)$	Create c leaf nodes
Build Huffman tree	$O(c \log c)$	Extract min ($2\times$) and insert, $c-1$ times
Generate codes	$O(c)$	Tree traversal
Encode file	$O(n \log c)$	For each char, append code of avg length $\log c$
Write header	$O(1)$	Constant 256 integers

$$T(n) = O(n) + O(c) + O(c \log c) + O(n \log c) + O(1)$$

$$T(n) = O(n + c \log c + n \log c)$$

$$\mathbf{T(n) = O(n \log c)}$$
 [assuming $c \ll n$]

Special cases:

- Worst case: $c = 256$, so $T(n) = O(n \log 256) = O(8n) = \mathbf{O(n)}$
- Best case: $c = 1$, $T(n) = \mathbf{O(n)}$

Space Complexity

- Frequency table: $O(256) = \mathbf{O(1)}$
- Huffman tree: **$O(c)$** nodes
- Code table: $O(256) = \mathbf{O(1)}$
- Bit buffer: $O(\log n)$ worst case
- **Total: $O(c) = O(1)$** since $c \leq 256$

3.2 Decompression Algorithm

Implementation: `huffman_decompress.cpp`

Algorithm Steps

1. Read frequency table from compressed file header (1024 bytes)
2. Read total character count
3. Rebuild Huffman tree using same algorithm as compression
4. Decode compressed data:
 - Read bits one at a time from compressed data
 - Traverse tree: left for 0, right for 1
 - When reaching leaf node: output character, return to root
 - Stop after decoding exactly n characters (avoids padding)

Time Complexity Analysis

- Read header and rebuild tree: $O(c \log c)$
- Decode n characters:
 - Each character requires traversing from root to leaf
 - Average depth: $O(\log c)$
 - Total: $O(n \log c)$ average

$$T(n) = O(c \log c) + O(n \log c)$$
$$T(n) = O(n \log c) \text{ [assuming } n \gg c]$$

Experimental Results

File Type	Original Size	Compressed Size	Ratio	md5 Verified
Small text	383 bytes	1249 bytes	326%	✓ Match
Medium text	1956 bytes	2147 bytes	110%	✓ Match
Large text	220 KB	126 KB	57%	✓ Match

Key Observations

- ⚠ Small files become larger due to 1KB header overhead
- ✓ Compression improves significantly for files > 3KB
- ✓ Achieved 43% compression ratio on large files
- ✓ All decompressed files match original (verified by md5sum)
- ✓ Compression is truly lossless - perfect reconstruction

Improvements and Optimizations

1. Reduce Header Overhead

- Store only non-zero frequencies (sparse encoding)
- Use variable-length integer encoding for frequencies
- Could reduce header from 1KB to ~100 bytes for typical text
- Complexity remains $O(n)$, but constant factor improves

2. Adaptive Huffman Coding

- Build tree dynamically while reading file
- No need to store frequency table (eliminates header)
- Single-pass encoding: $O(n \log c)$ time, $O(c)$ space
- Better for streaming data or unknown input size

3. Combine with Other Techniques

- Pre-process with Run-Length Encoding (RLE) for repetitive data
- Use Burrows-Wheeler Transform (BWT) before Huffman
- This is how bzip2 achieves better compression than gzip
- Complexity increases to $O(n \log n)$ due to BWT sorting

4. Canonical Huffman Codes

- Store only code lengths instead of full tree

- Reconstruct codes algorithmically
- Reduces header size significantly
- Used in DEFLATE (ZIP, gzip)

Summary and Conclusions

Implementation Summary

Program	Time	Space	Best For
scheduling.cpp Basic Algorithm 4.4	$O(n^2 \log n)$	$O(n)$	Small n , simplicity
scheduling_disjoint.cpp Optimized version	$O(n \log n)$	$O(n+d)$	Large n , performance
huffman_compress.cpp Compression	$O(n \log c)$	$O(1)$	Text compression
huffman_decompress.cpp Decompression	$O(n \log c)$	$O(1)$	Fast decompression

Key Learnings

1. Greedy Algorithms

- Can be optimal when problem has optimal substructure
- Require proof of correctness (greedy choice property)
- Simple but not always efficient

2. Data Structures Matter

- Disjoint sets dramatically improve efficiency
- From $O(n^2 \log n)$ $\rightarrow$ $O(n \log n)$: n -fold improvement
- Right structure for right problem is crucial

3. Asymptotic Analysis

- Reveals scalability of algorithms
- Constant factors matter for small inputs
- Growth rate dominates for large inputs

4. Compression Algorithms

- Huffman coding: optimal for prefix-free codes
- Header overhead significant for small files
- Trade-off between compression ratio and complexity

5. Implementation Details

- Bit-level manipulation requires care
- Edge cases (padding, empty files) are critical
- Testing and verification essential

Verification and Testing

All implementations have been thoroughly tested:

- ✓ **Scheduling: Verified against Table 1.1 from assignment**
- ✓ **Both scheduling approaches produce identical optimal results (profit = 170)**
- ✓ **Huffman: Tested on files ranging from 383 bytes to 220 KB**
- ✓ **Decompression: Verified using md5sum checksums**
- ✓ **All test cases pass without errors**
- ✓ **Edge cases: Empty files, single character, all same character**

References

1. Course textbook: *Algorithm Design and Analysis*
 2. Algorithm 4.4: Scheduling with Deadlines
 3. Appendix C: Disjoint Set Data Structure III
 4. D.A. Huffman, "A Method for the Construction of Minimum-Redundancy Codes", *Proceedings of the IRE*, 1952
 5. Brassard, G. and Bratley, P., "Algorithms: Theory and Practice", 1988
 6. Testing and verification assisted by Claude Code AI
-

END OF DOCUMENT

*All algorithms implemented, tested, and verified
Ready for submission*